

BLE Beacon — Missing MACs in POST

Root cause analysis & fix · Files reviewed: `beacon_configure.sh`, `beacon_post.sh`, `beacon_daemon.sh`, `beacon_ctl.sh`

BUG

root cause

WARN

contributing issue

NOTE

design observation

FIX

patched code

Symptom

Tracked MACs are *not all* appearing in the POST to the orchestrator. Some MACs go missing after a configure run or after the auto-restart loop kicks in; the device falls back to heartbeat (`i[0]{b,r}:`) even though `tracked_macs.txt` still lists the MACs.

Root cause

BUG

`cmd_start` in **`beacon_ctl.sh`** brings microcom back up but never re-loads the tracked MAC list into it. The MACs are sent to microcom *only once* — by the daemon’s feeder subshell at boot. After that, every restart path leaves microcom blind:

Restart path	microcom state	MAC list inside microcom
Daemon initial boot	started by daemon, feeder pushes ADDs from <code>tracked_macs.txt</code>	loaded
Auto-restart loop in daemon (<code>is_microcom_running</code> false)	calls <code>beacon_ctl.sh start</code> → <code>cmd_start</code>	EMPTY
<code>beacon_configure.sh</code> (stop → add/del → start)	stop, then <code>cmd_start</code> at end	EMPTY
User runs <code>beacon_ctl.sh restart</code>	<code>cmd_stop</code> + <code>cmd_start</code>	EMPTY

Once microcom has zero MACs to track, no `MATCHED:` lines are emitted, `/tmp/beacon_rssi.log` stops growing, and within 60 s the post script’s cutoff ages out the old entries → POST goes to the heartbeat path with no MACs.

Where the bug lives

`beacon_ctl.sh` – `cmd_start()`

```

cmd_start() {
    rm -f "$MANUAL_STOP_FLAG"
    if is_microcom_running; then
        log_msg WARN "microcom already running"
        return
    fi
    if [ "$(check_daemon)" = "stopped" ]; then
        log_msg INFO "Daemon not running, starting daemon..."
        sh "$DAEMON_SCRIPT" > /dev/null 2>&1 &
        sleep 3
        if [ "$(check_daemon)" = "running" ]; then
            log_msg OK "Daemon started"
        else
            log_msg ERROR "Daemon failed to start"
            return
        fi
    fi
    if [ ! -p "$FIFO_PIPE" ]; then
        rm -f "$FIFO_PIPE"
        mkfifo "$FIFO_PIPE" 2>/dev/null
        log_msg INFO "Recreated FIFO pipe"
    fi
    rm -f /var/lock/LCK..ttyS1
    (
        microcom -s 115200 "$UART_PORT" < "$FIFO_PIPE" 2>&1 | while IFS= read -r line; do
            if echo "$line" | grep -q "MATCHED:"; then
                MAC=$(echo "$line" | sed -n 's/.*MATCHED:[[:space:]]*\\([0-9A-Fa-f:]*\\).*\\1/p')
                RSSI=$(echo "$line" | sed -n 's/.*RSSI:[[:space:]]*\\(-\\?[0-9]*\\).*\\1/p')
                [ -n "$MAC" ] && [ -n "$RSSI" ] && echo "$(date '+%Y-%m-%d %H:%M:%S')|$MAC|$RSSI|$(da
            fi
        done
    ) &
    sleep 2
    # <<< BUG: microcom is up here but NO 'ADD <mac>' commands are sent. >>>
    # <<< The previous microcom consumed the feeder's initial ADDs; >>>
    # <<< FIFOs aren't replayable, so the new microcom tracks ZERO MACs. >>>

    if is_microcom_running; then
        log_msg OK "microcom started"
    else
        log_msg ERROR "microcom failed to start"
    fi
}

```

beacon_configure.sh – stop / add / start bracket

```
# Stop microcom before making changes
/usr/sbin/beacon_ctl.sh stop 2>/dev/null
# <<< WARN: this kills microcom (the FIFO reader). Subsequent      >>>
# <<< 'beacon_ctl.sh add/del' calls do `printf ... > $FIFO_PIPE`,    >>>
# <<< opening the FIFO for write with NO READER -> that open() blocks. >>>
# <<< Even if it doesn't fully hang, the data flow is fragile.      >>>

...
    if beacon_ctl.sh add "$MAC_UPPER" >/dev/null 2>&1; then
        ...
    fi
...
/usr/sbin/beacon_ctl.sh start 2>/dev/null
# <<< This restart triggers the cmd_start bug above:                >>>
# <<< microcom comes back tracking 0 MACs.                          >>>
```

beacon_ctl.sh – cmd_add_mac() / cmd_del_mac()

```
cmd_add_mac() {
    ...
    echo "$MAC" >> "$MAC_LIST_FILE"
    log_msg OK "Added $MAC to active tracking list"

    if [ -p "$FIFO_PIPE" ]; then
        printf "ADD %s\r\n" "$MAC" > "$FIFO_PIPE" 2>/dev/null
        # <<< WARN: when called from beacon_configure.sh after a 'stop', >>>
        # <<< microcom is dead -> this open(O_WRONLY) blocks/stalls.    >>>
        log_msg OK "Sent ADD command to daemon"
    fi

    return 0
}
```

beacon_post.sh – MAC_COUNT calc

```
MAC_COUNT=$(echo "$BEACON_DATA" | grep -c '^ ' 2>/dev/null || echo "0")
# <<< WARN: when no matches, grep -c PRINTS "0" AND returns exit 1.    >>>
# <<< The '|| echo "0"' fires anyway, so MAC_COUNT becomes "0\n0",    >>>
# <<< which breaks the later [ "$MAC_COUNT" -eq 0 ] arithmetic test.  >>>
# <<< (Only matters when BEACON_DATA is non-empty but has no '^ '    >>>
# <<< lines – the empty-string case is short-circuited earlier.) >>>
```

Fix — reload MACs into microcom on every start

beacon_ctl.sh – patched cmd_start()

```

cmd_start() {
    rm -f "$MANUAL_STOP_FLAG"
    if is_microcom_running; then
        log_msg WARN "microcom already running"
        return
    fi
    if [ "$(check_daemon)" = "stopped" ]; then
        log_msg INFO "Daemon not running, starting daemon..."
        sh "$DAEMON_SCRIPT" > /dev/null 2>&1 &
        sleep 3
        if [ "$(check_daemon)" = "running" ]; then
            log_msg OK "Daemon started"
            return # FIX: daemon's own feeder already pushed ADDs –
                  # don't double-load.

        else
            log_msg ERROR "Daemon failed to start"
            return
        fi
    fi
    if [ ! -p "$FIFO_PIPE" ]; then
        rm -f "$FIFO_PIPE"
        mkfifo "$FIFO_PIPE" 2>/dev/null
        log_msg INFO "Recreated FIFO pipe"
    fi
    rm -f /var/lock/LCK..ttyS1
    (
        microcom -s 115200 "$UART_PORT" < "$FIFO_PIPE" 2>&1 | while IFS= read -r line; do
            if echo "$line" | grep -q "MATCHED:"; then
                MAC=$(echo "$line" | sed -n 's/.*MATCHED:[[:space:]]*([0-9A-Fa-f:]*\).*\/\1/p')
                RSSI=$(echo "$line" | sed -n 's/.*RSSI:[[:space:]]*(-\?[0-9]*\).*\/\1/p')
                [ -n "$MAC" ] && [ -n "$RSSI" ] && \
                    echo "$(date +%Y-%m-%d %H:%M:%S)|$MAC|$RSSI|$(date +%s)" >> "$RSSI_LOG"
            fi
        done
    ) &
    sleep 2
    if ! is_microcom_running; then
        log_msg ERROR "microcom failed to start"
        return
    fi

    # FIX: reload tracked MACs into the freshly started microcom.
    # microcom is the FIFO reader now, so writes won't block.
    if [ -s "$MAC_LIST_FILE" ]; then
        COUNT=0
        while read -r mac; do
            [ -z "$mac" ] && continue
            printf "ADD %s\r\n" "$mac" > "$FIFO_PIPE" 2>/dev/null
            COUNT=$((COUNT + 1))
            sleep 1
        done < "$MAC_LIST_FILE"
        printf "START\r\n" > "$FIFO_PIPE" 2>/dev/null
        log_msg OK "Reloaded $COUNT MACs into microcom"
    fi

    log_msg OK "microcom started"
}

```

Why the early `return` matters

When `cmd_start` is invoked while the daemon is *down*, it boots the daemon — and the daemon's own startup feeder already pushes `ADD` commands from `tracked_macs.txt`. Letting the function continue past that point would double-add every MAC.

Why FIFO writes are safe in the patch

The new `ADD-replay` loop runs *after* `microcom` has been started in the background subshell, so `microcom` is the FIFO reader. `open(O_WRONLY)` succeeds without blocking. The same is **not** true of the writes that `beacon_configure.sh` performs between its `stop` and `start` — those happen with no reader and should be removed (see follow-up below).

How to verify the diagnosis

After running `beacon_configure.sh` (or `beacon_ctl.sh restart`), check whether new entries are still being appended to `/tmp/beacon_rssi.log`:

```
wc -l /tmp/beacon_rssi.log
sleep 30
wc -l /tmp/beacon_rssi.log
```

- **Count not growing** → `microcom` isn't tracking anything → this is the right fix.
- **Count grows for some MACs only** → not a `microcom`-reload issue. Either (a) those beacons aren't in range / advertising often enough, or (b) the 60-second cutoff in `beacon_post.sh` is too tight for their advertise interval.

Follow-ups (optional, not strictly required for the symptom)

- **WARN** Drop the `stop / start` bracket in `beacon_configure.sh`. `beacon_ctl.sh add / del` are designed to push to a *live* FIFO; stopping `microcom` around them is harmful (FIFO write blocks) and unnecessary once `cmd_start` reloads MACs from the file.
- **WARN** Replace `MAC_COUNT=$(... grep -c '^ ' || echo "0")` with `MAC_COUNT=$(printf '%s\n' "$BEACON_DATA" | grep -c '^ ')`. `grep -c` already prints `0` on no-match; the `||` appends a second `0`.
- **NOTE** The comment in `beacon_post.sh` says "Averages last 1 minute of data" but the `awk` only keeps the *latest* RSSI per MAC (`last_rssi[mac_lower] = rssi`). Either update the comment or implement the average.